

NovaMart Customer Complaint Prioritizer: A Gen AI Proof of Concept

Derrick Cummings and Cris Dizon

Department of Computer Information Systems, Georgia State University

CIS 8600: Generative AI

September 30, 2026

Executive Summary

Business Problem

NovaMart, a Seattle-based e-commerce retailer, processes approximately 800 to 1,200 customer complaints daily through a largely manual, first-come, first-served system. Without urgency-based prioritization, critical complaints may experience unnecessary delays. The company's average complaint response time is 18 hours, compared with its strategic target of four hours. These challenges contribute to customer dissatisfaction, operational inefficiencies, and retention concerns.

Industry Background

NovaMart operates in a competitive e-commerce environment where timely complaint resolution is essential to customer satisfaction and retention. Although the company performs competitively in areas such as product quality and delivery, it falls below industry benchmarks in customer-service responsiveness, complaint resolution, returns, refunds, and communication during delays.

Solution Approach

The project team developed the NovaMart Customer Complaint Prioritizer, a generative AI proof of concept designed to classify complaint urgency, analyze customer sentiment, and generate professional response drafts for human review. The application uses Anthropic's Claude Haiku 4.5 through Amazon Bedrock and integrates a browser-based interface, serverless processing, API connectivity, and operational monitoring.

Prototype Scope and Future Production Capabilities

The prototype demonstrates core complaint-analysis functionality but does not include production-grade model adaptation, comprehensive feedback mechanisms, or automated complaint resolution. The report explores potential future capabilities, including supervised fine-tuning, reinforcement learning from human feedback, a production feedback loop, and a controlled agentic workflow. These enhancements would require approved data, enterprise-system integrations, governance controls, and human oversight.

Solution Implementation

The solution was implemented using Amazon Bedrock to access Claude Haiku 4.5, AWS Lambda to process complaints, Amazon API Gateway to route requests, Amazon S3 to host the web interface, and Amazon CloudWatch to monitor application operations. The resulting application successfully accepts fictional customer complaints and returns urgency classifications, sentiment assessments, and response drafts through an interactive interface.

Challenges

Implementation challenges included AWS region restrictions, inconsistent urgency classifications, API connectivity and CORS errors, and integration across multiple application components. The project team addressed these issues through configuration changes, prompt refinement, troubleshooting, and repeated testing. The completed proof of concept demonstrated a functioning end-to-end application, although actual improvements in customer-service response times and business outcomes were not measured.

Business Problem

NovaMart, a Seattle-based e-commerce retailer with 450 employees, receives between 800 and 1,200 customer complaints daily. The current customer service system relies on spreadsheet-based complaint intake and ticketing and does not integrate with cloud-based artificial intelligence capabilities. As a result, the company is unable to prioritize complaints by urgency, leading to all complaints being processed in a first-come, first-served queue.

NovaMart's customer service operation faces three interconnected challenges:

- No prioritization capability. All complaints enter a manual queue, causing urgent issues such as safety concerns, legal threats, or suspected fraud to wait behind routine inquiries. There is no system to flag critical issues for immediate escalation.
- Slow response times. The average complaint response time is 18 hours, compared to the industry average of 12 hours (MacDonald, 2023). NovaMart has established a strategic objective to reduce response time to 4 hours. Achieving this target is challenging under the current manual process without improved triage efficiency or increased staffing.
- Below-industry customer satisfaction. Customer feedback is below the industry midpoint in five areas directly tied to complaint handling: customer service responsiveness, complaint resolution time, return process, refund speed, and communication during delays.

These three customer service issues have resulted in the following broader business impacts for NovaMart:

- Retention rate: 58% (below industry average)

- New customer growth: 22% annually (but losing existing customers)
- Agent turnover: 14% annually
- Brand damage from negative reviews and social media complaints

To address these problems, the proposed solution must provide three core capabilities:

- Sentiment analysis: Identifies whether the customer's tone is angry, frustrated, or neutral.
- Urgency classification: Assigns High, Medium, or Low priority based on the issue's risk and customer impact.
- Response generation: Creates a short, professional draft response for agent review.

Traditional text-classification models are capable of assigning predefined labels such as sentiment or urgency ("Introduction – Text Classification," n.d.). However, these models generally require labeled training data and separate models or rule sets for each task.

Furthermore, they do not generate agent-reviewed response drafts ("Introduction – Text Classification," n.d.).

Implementing generative AI to address these business impacts would directly support NovaMart's strategic goals:

- Modernize customer-facing technologies.
- Reduce complaint response time from 18 hours to 4 hours.
- Improve public image and brand to attract new customers.
- Improve employee retention by reducing manual workload.

In the absence of generative AI implementation, NovaMart would be required to hire a substantially larger number of agents to achieve the same response time. This approach would increase operational costs while failing to address the underlying prioritization issue.

Industry Background

NovaMart operates in a high-volume, competitive e-commerce environment. The company serves consumers, small businesses, and Prime-like subscribers across 12 product categories, processing 15,000 to 45,000 orders and 800 to 1,200 customer complaints each day. Timely support for delivery issues, returns, refunds, account concerns, and safety-related problems directly affects customer satisfaction, retention, employee workload, and NovaMart's competitive position.

E-commerce operations require reliable processes for purchasing, delivery, returns, refunds, and customer communication. Complaints vary in severity, from routine shipping inquiries to food-safety, account-security, and urgent financial concerns. Effective customer service must distinguish between routine cases and those requiring prompt escalation.

NovaMart competes with major retailers such as Amazon, Walmart, Target, Best Buy, and Wayfair. Although the company meets or exceeds industry benchmarks for product quality, website usability, payment security, delivery speed, and packaging, it performs below the industry midpoint in customer-service responsiveness, complaint resolution time, returns, refund speed, and communication during delays.

NovaMart employs 45 customer-service agents within a workforce of 450 employees. Its early Zendesk system and spreadsheet-based complaint process are not well suited to the volume and urgency of customer complaints it receives. Improving complaint handling is therefore

important to customer experience, employee workload, brand reputation, and the company's modernization efforts.

Solution Approach

The NovaMart Customer Complaint Prioritizer is a generative-AI proof of concept designed to enable agents to rapidly identify urgent complaints. The solution comprises five integrated components: a pretrained Claude Haiku 4.5 language model, a Lambda complaint-analysis function, an API Gateway connection, an S3-hosted web page, and a monitoring and logging service. Together, these components move a complaint from agent submission to AI analysis, human review, and operational monitoring.

Large Language Model (LLM): Claude Haiku 4.5

Anthropic's Claude Haiku 4.5 serves as the pretrained large language model (LLM) for the proof of concept. It processes each customer complaint and generates an urgency classification, sentiment analysis, and draft response for agent review. During the Experiment and Select phase of the Generative AI Project Life Cycle, the project team compared Claude Haiku 4.5 with Claude Sonnet 5, another Anthropic model designed for coding, agents, and professional-scale work ("Claude Sonnet 5," n.d.). The comparison considered latency, input and output pricing, context window, maximum output, and overall model positioning (Fregly et al., 2023, Chapter 1).

Table 1*Comparison of Claude Haiku 4.5 and Claude Sonnet 5*

Metric	Claude Haiku 4.5	Claude Sonnet 5
Latency	Fastest	Fast
Input price	\$1 / million tokens	\$2 / million tokens
Output price	\$5 / million tokens	\$10 / million tokens
Context window	200k tokens	1 million tokens
Maximum output	64k tokens	128K tokens
Model positioning	Fastest model with near-frontier intelligence	Combination of speed and intelligence

Note. Prices are listed per million tokens. Both models were available through Amazon Bedrock.

The project team selected Claude Haiku 4.5 because NovaMart needs fast, lower-cost analysis of short customer complaints. Faster responses are important when agents are waiting for complaint analysis, and Haiku 4.5 processes complaint text at half the input and output cost of Claude Sonnet 5 (“Claude Haiku 4.5,” n.d.; “Claude Sonnet 5,” n.d.). Its 200K context window and 64K maximum output are sufficient for NovaMart’s short prompts and complaint submissions (“Claude Haiku 4.5,” n.d.; “Claude Sonnet 5,” n.d.). The model therefore meets the prototype’s requirements for rapid, cost-effective urgency classification, sentiment analysis, and draft-response generation without the additional cost or complexity of a larger model (“Haiku by Anthropic,” 2025; “Haiku 4.5 - Amazon,” n.d.).

During the Adapt, Align, and Augment phase, the project team developed and implemented a system prompt that instructs Claude to return urgency, sentiment, and an agent-reviewed draft response. The prompt also includes High, Medium, and Low urgency definitions

and examples for each level (Fregly et al., 2023, Chapter 1). These instructions align Claude's output with NovaMart's complaint-prioritization requirements.

During the Evaluate phase, the project team tested Claude using fictional complaints and compared the outputs with expected labels. The testing determined whether the model classified urgency and sentiment appropriately and generated safe, professional draft responses for agent review (Fregly et al., 2023, Chapter 1).

Lambda Complaint-analysis Function

The Lambda complaint-analysis function serves as the proof of concept's complaint processor. It receives a customer complaint, sends the system prompt and complaint to Claude Haiku 4.5 through Amazon Bedrock, and returns the analysis. Its integration represents the Deploy and Integrate phase of the Generative AI Project Life Cycle (Fregly et al., 2023, Chapter 1). The function follows a request-to-response sequence:

- **Configure the function and model connection.** The imports (json, boto3, and ClientError) support JSON processing, AWS service calls, and Bedrock-related error handling. The Bedrock client configuration connects Lambda to Amazon Bedrock Runtime in us-east-1. The model ID identifies Claude Haiku 4.5, and the system prompt defines Claude's role, required JSON output fields, urgency definitions and examples, and response rules.
- **Receive and validate the complaint.** The `lambda_handler(event, context)` function runs when Lambda receives either a test event or an API Gateway request. It reads the complaint from the appropriate request-body format, removes extra spaces, and returns a 400 error if no complaint was submitted or if the request body is not valid JSON.

- **Send the complaint to Claude and extract the result.** The `bedrock.converse(...)` request sends the system prompt and customer complaint to Claude through Bedrock. It uses a maximum response length and a low temperature to support consistent results. The function then extracts Claude's generated text from the Bedrock response.
- **Return the analysis or an error.** When processing succeeds, the function returns the model's analysis as JSON with a 200 status code so that API Gateway and the web page can receive it. If the Bedrock request or another AWS operation fails, the function returns a 500 error.

Figure 1

NovaMart Lambda Function Code

```
lambda_function.py
6
7  bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")
8
9  MODEL_ID = "us.anthropic.claude-haiku-4-5-20251001-v1:0"
10
11  SYSTEM_PROMPT = """
12
13  You are a customer complaint prioritizer.
14
15  Read the customer complaint and return only valid JSON with:
16  - urgency: High, Medium, or Low
17  - sentiment: Angry, Frustrated, or Neutral
18  - draft_response: a short, professional response for a customer-service agent to review
19  Classify urgency using these definitions:
20
21
22  High: Use only when the complaint involves immediate safety, health, food-safety, account-security, fraud or unauthorized access, a duplicate charge creating an i
23  Safety hazard: The space heater I bought yesterday started smoking after 20 minutes of use. I unplugged it immediately because I am worried it could cause a f
24  Health concern: My daughter ate one of the snack bars I purchased and started having an allergic reaction. The package did not clearly list the ingredient we
25  Food safety: I picked up a grocery order this afternoon, and the milk and chicken were warm when I got home. I do not feel safe using them.
26  Account security: I received an order confirmation for an order I never placed. When I checked my account, my delivery address had been changed.
27  Fraud/unauthorized access: There are purchases on my account that I did not make. I believe someone has used my account without permission.
28  Duplicate charge: I was charged twice for the same order, and the duplicate charge has affected the money available in my bank account.
29  Persistent serious issue: I have contacted support three times because the charging cable I received repeatedly becomes extremely hot while charging my device
30  Medium: Use when the problem has a meaningful customer impact but does not involve immediate safety, security, or serious financial risk. Examples include a damag
```

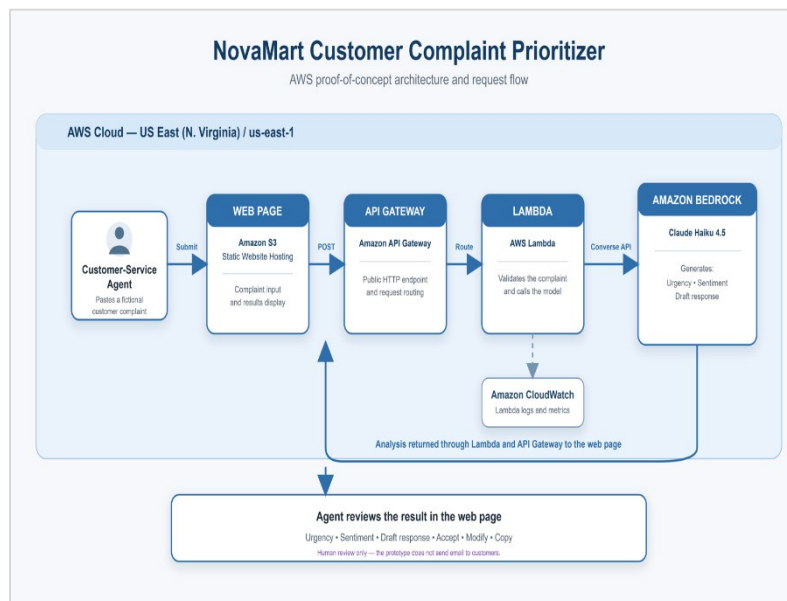
Note. Excerpt showing the Bedrock connection, Claude Haiku 4.5 model ID, and system prompt.

API Gateway

The API Gateway serves as the proof-of-concept's public web endpoint and request-routing component. It provides the URL that receives requests from the web page and routes them to the Lambda complaint-analysis function. The API Gateway includes Cross-Origin Resource Sharing (CORS) configuration, which allows the browser-based web page to send requests to the API and receive results ("Configure CORS," n.d.). Integrating the API Gateway reflects the Deploy and Integrate phase of the Generative AI Project Life Cycle because it connects the user-facing web page to the Lambda function (Fregly et al., 2023, Chapter 1). For example, when a customer-service agent pastes a fictional complaint into the web page and selects Analyze, the API Gateway receives the request and passes it to Lambda for analysis. The Lambda function then invokes Claude Haiku 4.5 through Amazon Bedrock and returns the analysis to the web page.

Figure 2

API Gateway Request Routing



Note. NovaMart AWS architecture and request flow.

Web Page

The web page serves as the proof-of-concept user interface, allowing an agent to paste a customer complaint, select Analyze, and view urgency, sentiment, and a draft response. It consists of front-end code that runs in the user's browser. The HTML creates the page structure and input/result sections. The CSS controls the visual design and urgency colors. The JavaScript sends the complaint to API Gateway, displays the returned analysis, and provides Accept, Modify, and Copy review controls.

Its integration reflects the Deploy and Integrate phase of the Generative AI Project Life Cycle because the web page provides a user-facing way to submit a customer complaint and view Claude's output (Fregly et al., 2023, Chapter 1). For the prototype, the web page lets the project team demonstrate the solution from a graphical user interface (GUI) and customer-service-agent perspective rather than only showing a Lambda test. A Lambda test shows a technical AWS console view: the user enters the complaint in a JSON test-event box, selects Test, and then sees the returned response as raw JSON with execution details such as status code, duration, memory use, and logs.

Figure 3

NovaMart Web Interface

NovaMart Customer Complaint Prioritizer

Paste a fictional customer complaint below.

Customer complaint

My package was supposed to arrive three days ago but tracking hasn't updated. I need this for a birthday party this weekend. Can you tell me where it is? I've been a customer for two years and this is the second time this has happened.

Analyze

Result

Urgency: Medium

Sentiment: Frustrated

Draft response for agent review

Thank you for reaching out, and I sincerely apologize for the delay with your delivery. I understand how frustrating this is, especially with the birthday party this weekend. I'd like to help locate your package right away. Could you please provide your order number so I can investigate the current status and tracking information? We'll work to resolve this as quickly as possible.

Accept Draft

Modify Draft

Copy Final Response

Review the AI-generated draft before using it. It has not been sent to any customer.

The draft is for human review only. This prototype does not send messages to customers.

Note. The page submits a customer complaint and displays the analysis for human review.

Figure 4

NovaMart Web Interface Front-end Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>NovaMart Customer Complaint Prioritizer</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 40px auto;
      padding: 0 20px;
      background: #f7f8fa;
      color: #1f2937;
    }
    .container {
      background: white;
      border: 1px solid #d0d7de;
      border-radius: 10px;
      padding: 28px;
    }
    h1 {
      color: #12355b;
    }
    textarea {
      width: 100%;
      min-height: 125px;
      padding: 10px;
      margin: 8px 0 16px;
      box-sizing: border-box;
      border: 1px solid #aeb8c2;
      border-radius: 6px;
      font-family: Arial, sans-serif;
      font-size: 15px;
    }
    button {
      padding: 9px 14px;
      margin: 4px 6px 10px 0;
      border: none;
      border-radius: 5px;
    }
```

Note. This code defines the web page's structure and visual design.

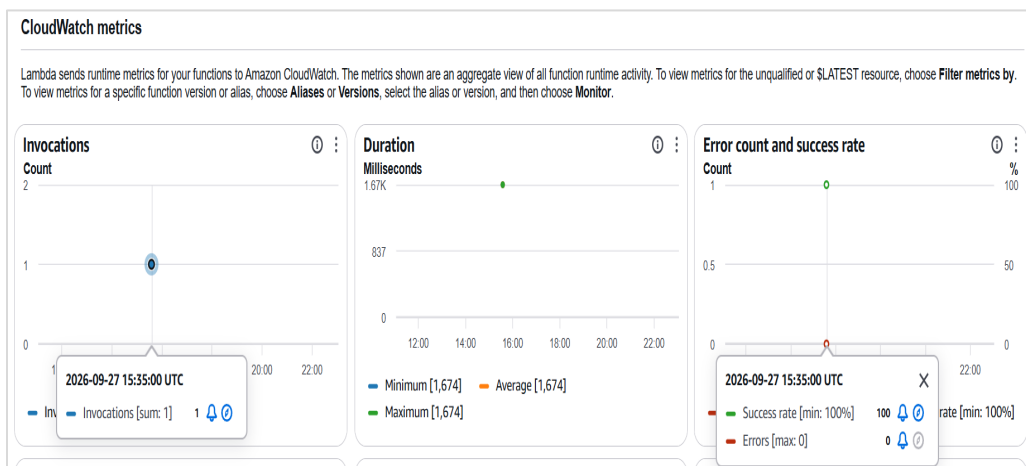
Monitoring and Logging Service

The monitoring and logging service records Lambda activity for the proof of concept. It tracks the number of Lambda invocations (calls), each analysis's duration, error count and success rate, throttling activity, and total concurrent executions. It also automatically stores Lambda execution logs, including technical records such as the Request ID—a unique ID for each run—memory size (allocated to Lambda), and maximum memory used.

The monitoring and logging service's integration reflects the Monitor phase of the Generative AI Project Life Cycle because it records Lambda execution activity and performance (Fregly et al., 2023, Chapter 1). For the prototype, it allows the project team to provide operational evidence that the Lambda function executed, determine analysis duration and errors, and retain execution logs for review.

Figure 5

Monitoring and Logging Service Metrics



Note. The metrics show one Lambda invocation, its execution duration, and a 100% success rate with no errors.

Figure 6

Monitoring and Logging Service Log

Lambda logs all requests handled by your function and automatically stores logs generated by your code through Amazon CloudWatch Logs. To validate your code, instrument it with custom logging statements. The following tables list the most recent and most expensive function invocations across all function activity. To view logs for a specific function version or alias, visit the **Monitor** section at that level.

Recent invocations	
@entity.KeyAttributes.Name	cis8600-complaint-prioritizer-analyze
@entity.KeyAttributes.Type	Service
@ingestionTime	1790523319550
@initDuration	516.95
@log	959015415226:/aws/lambda/cis8600-complaint-prioritizer-analyze
@logGroupId	b71b1cc0-d86d-477d-bc78-9a1a90b6a2bf
@logStream	2026/09/27/[\$LATEST]f7e2f805aab489281928b8b5ea27937
@logStreamId	b71b1cc0-d86d-477d-bc78-9a1a90b6a2bf::b265f6e57171e5aa72b04f2682766ea4c3957ec799c47c9dc7753c582c230c8d::1790523319542
@maxMemoryUsed	9.2E7
@memorySize	1.28E8

Note. The log records a recent Lambda invocation, including the log stream, initialization duration, memory allocation, and maximum memory used.

Prototype Scope and Future Production Capabilities

The proof of concept demonstrates use-case identification, experimentation and selection, deployment and integration, and monitoring. The following sections describe production capabilities that were not feasible to complete and how they might be followed in a real-world implementation.

Model Pretraining

The project team selected the pretrained Claude Haiku 4.5 model because training a model from scratch requires massive datasets, substantial time, and significant computing resources. These requirements would exceed the project's scope, timeline, resources, budget, and risk level. Therefore, during the Experiment and Select phase of the Generative AI Project Life Cycle, the project team chose an existing pretrained model rather than pursuing pretraining. In a real-world implementation, model pretraining would be done by a foundation-model provider

rather than NovaMart. Typically, model pretraining requires the following activities (Chaumond, 2020):

- Find a dataset: The first pretraining step, which requires the organization to find and combine large, diverse text datasets. Larger and more varied datasets generally produce better pretrained-model results.
- Train a tokenizer: The second step is to train a tokenizer on the organization's text dataset so it can convert text into tokens the model can process. The tokenizer creates and saves vocabulary and token-merge files and is tailored to represent the language or domain data efficiently.
- Train a language model from scratch: The third step is to configure a training tool without an existing model checkpoint, select a model type and learning objective, load the tokenized text dataset, and choose training settings such as learning rate, number of epochs, batch size, and checkpoint frequency. Afterward, run training and evaluation, then monitor the results with a tool such as TensorBoard.
- Check that the LM actually trained: The fourth step is to review training and evaluation loss results, then test the model with prompts containing masked tokens. The model should return likely replacements and their probabilities, which you can review to assess whether it learned language patterns and context.
- Fine-tune your LM on a downstream task: The fifth step is to use an annotated dataset for a specific task and train the pretrained model for that task. The organization should review training and evaluation results, then use a task-specific pipeline to test the model's predicted labels and confidence scores.

- Share your model: The sixth step is to document and release the completed model for approved use. The organization should save the model in a controlled repository and create a model card describing the model, training data, preprocessing steps, training settings, evaluation results, intended uses, limitations, and known risks.

Model Fine-tuning

Fine-tuning was not feasible for the project prototype as part of the Adapt, Align, and Augment phase of the Generative AI Project Life Cycle because NovaMart does not have approved, prepared historical complaint data for training. In a real-world implementation, NovaMart would first obtain protected complaint records and create a custom instruction dataset. Each training example would pair a customer complaint and instruction with a verified expected output, such as the correct urgency level, sentiment label, and an agent-approved response (Fregly et al., 2023, Chapter 5).

The organization would divide the data for training and evaluation, select a fine-tunable model, and run supervised fine-tuning (Fregly et al., 2023, Chapter 5). During this process, the model generates an output, compares it with the verified human-created output, calculates the difference, and updates its model weights over repeated training rounds (Fregly et al., 2023, Chapter 5). NovaMart would establish baseline evaluation metrics before fine-tuning and compare results before and after training to determine whether the model improved without creating unacceptable safety or quality issues.

Reinforcement Learning from Human Feedback (RLHF)

RLHF was not feasible for the project prototype during the Adapt, Align, and Augment phase of the Generative AI Project Life Cycle because it requires structured human feedback, a

reward-model process, substantial time, and additional technical resources. In a real-world implementation, NovaMart would first define human-preference criteria for complaint responses, including helpfulness, honesty, harmlessness, professional tone, and appropriate escalation (Fregly et al., 2023, Chapter 7). Customer-service experts would then review multiple model-generated responses to the same complaint and rank them from most to least preferred (Fregly et al., 2023, Chapter 7). NovaMart would establish clear review instructions and quality controls to ensure consistent application of these criteria.

The ranked responses would be converted into preference data and used to train a reward model that identifies which outputs best align with human reviewers' preferences (Fregly et al., 2023, Chapter 7). During reinforcement-learning fine-tuning, the reward model would guide updates to the generative model's weights, encouraging more preferred responses and discouraging less preferred responses (Fregly et al., 2023, Chapter 7). Before deployment, NovaMart would evaluate the updated model for safety, accuracy, and response quality.

Production Feedback Loop

The project includes basic technical monitoring but not a production feedback loop. In a real-world implementation, NovaMart would continuously monitor input and output quality, latency, failures, cost, data or model drift, and privacy or safety risks as part of the Monitor phase of the Generative AI Project Life Cycle ("Production ML Systems," n.d.; "Artificial Intelligence Risk," 2024). The organization would also collect customer-service-agent feedback and customer-outcome data to identify incorrect classifications, unsafe draft responses, and opportunities to improve the system prompt, retrieval data, or model configuration.

Agentic Workflow

Building on the prototype's analysis and monitoring capabilities, Phase 2 could extend the tool from an agent assistant to an agentic workflow designed for controlled resolutions. The workflow would retain urgency classification, sentiment analysis, response drafting, and prioritization while introducing limited automated actions for eligible cases. Rather than following a fixed sequence of automated steps, the AI agent would interpret each complaint, determine which information and tools are needed, evaluate permitted resolution options, and coordinate authorized actions through connected enterprise systems, subject to predefined policies and human oversight. Upon verification of customer identity, order details, return-window status, item eligibility, and prior interactions, the system could initiate an approved return, generate a shipping label, or issue a promotional code within predefined parameters.

Responsible execution of these actions would require recommendations to consider the customer's lifetime value (LTV), return history, previous refunds, prior compensation, support tickets, and unresolved issues. The system would clarify how approved LTV information influenced the offer rather than make financial predictions based solely on the complaint. A history of returns alone would not constitute evidence of abuse. However, if an active abuse or fraud flag exists, if data is missing or inconsistent, or if policy exceptions or requests exceed authorized limits, human review would be required. The assigned agent would receive the complaint, verified details, relevant history, actions already taken, and a policy-based recommendation for approval or modification.

The workflow would require integrations with customer, commerce, returns, support, risk, and policy systems to execute these decisions and actions. These systems would provide verified customer and order information, LTV scores, return and shipment records, prior support

interactions, fraud indicators, current rules, compensation limits, and approved promotion codes. This integration would ensure that the workflow operates strictly within approved policies and routes any exceptions to the appropriate agent or specialist.

Solution Implementation

The preceding sections describe how the prototype operates. The following sections identify the AWS services used to implement the prototype and explain why each service was selected.

Amazon Bedrock

Amazon Bedrock is an AWS-managed service that gives organizations secure access to foundation models so they can build and scale generative AI applications (“Overview – Amazon Bedrock,” n.d.). For the project, Amazon Bedrock was the managed AWS service used to access the Claude Haiku 4.5 pretrained model, where Lambda sent the system prompt and fictional complaint to Claude through Bedrock.

AWS Lambda

AWS Lambda is a serverless AWS service that runs application code without requiring the project team to manage servers, capacity, scaling, maintenance, or patching (“What is AWS Lambda?,” n.d.). In the NovaMart prototype, Lambda runs the complaint-analysis code when API Gateway sends a request and returns the result to the web page.

The project team also considered AWS Lambda MicroVMs, which provide persistent, isolated environments for workloads that require saved state, custom software, longer sessions, or multiple connections. Both options are managed AWS compute services; however, Lambda Functions are designed for short, automatically scaled request-response or event-driven tasks.

Lambda Functions were selected because NovaMart uses a short, independent workflow: an agent submits one complaint, Lambda sends it to Claude for analysis, and Lambda returns one result. This approach does not require session-state or MicroVM lifecycle management (“What is AWS Lambda?,” n.d.).

Amazon API Gateway

Amazon API Gateway is a managed AWS service that creates an API—a web address and set of rules that lets one application communicate with another (“Amazon API Gateway,” n.d.). In NovaMart, it is the middle connection between the web page and Lambda – it receives the complaint from the page and sends it to the Lambda function. This service is necessary because the web page cannot directly invoke the Lambda function; API Gateway provides the URL and routing layer that lets the browser submit a complaint to Lambda and receive the analysis result back (“Amazon API Gateway,” n.d.).

Amazon S3 Static Website Hosting

Amazon S3 static website hosting is an AWS service that stores website files and makes them available through a public website address (“Use Amazon S3,” n.d.). It delivers static files—such as HTML, CSS, JavaScript, and images—to a visitor’s browser, which then runs the front-end code and displays the web page (“Use Amazon S3,” n.d.). The service was necessary because it provided a simple, public location to host the NovaMart complaint-prioritizer web page. It let the project team open the page in a browser, enter a fictional complaint, and use the API Gateway URL built into the page to request analysis.

Amazon CloudWatch

Amazon CloudWatch is an AWS monitoring and logging service that provides visibility into AWS resources and applications (“Amazon CloudWatch”, n.d.). It automatically collects performance metrics and stores logs, allowing users to review activity such as Lambda invocations, execution duration, errors, success rate, memory use, and individual execution records (“Amazon CloudWatch”, n.d.). The service was necessary because the project team needed evidence that the complaint-analysis function ran successfully and a way to investigate technical issues. CloudWatch allowed the project team to verify completed Lambda requests, review execution details, and identify errors while testing the prototype.

Challenges

Building the NovaMart proof of concept required the project team to configure several connected AWS services, test the AI model’s responses, and troubleshoot issues that occurred during implementation. The following challenges describe the issues encountered, their causes, the corrective actions taken, and the lessons learned for a future real-world implementation.

AWS Region Restriction

An AWS region restriction is a security or governance control that limits where users and applications can create resources or run API operations within specific geographic areas (“Defining Boundaries,” n.d.). During the NovaMart project, the AWS console showed the Ohio Region (us-east-2) as restricted, so the project could no longer use that Region. The project team fixed this issue by changing the console region selector to N. Virginia (us-east-1) for the Amazon Bedrock, Lambda, API Gateway, CloudWatch, and the S3-hosted web page.

Bedrock access was confirmed in N. Virginia before continuing the prototype setup. This restored an accessible shared Region, allowing the web page, API Gateway, Lambda function, Claude model through Bedrock, and CloudWatch monitoring to work together. The lesson learned is that AWS resources are tied to specific regions and before creating project services, the project team should confirm that the selected region is accessible and supports all required services, or understand how to move AWS resources/projects to another region if one becomes unavailable after initial assignment.

Urgency Classification Inconsistency

Urgency classification inconsistency is a model's inability to consistently assign the expected urgency level based on risk and impact. During testing, Claude did not consistently assign the expected urgency. For example, Claude initially classified a complaint about waiting several weeks for a refund as Medium, while the project team expected High because the issue was persistent and involved the customer's money. The original system prompt told Claude to return High, Medium, or Low urgency but did not clearly define boundaries between levels. A delayed refund can appear as Medium unless the prompt explains when the delay becomes a persistent or serious financial issue. Claude also needed clearer guidance to base urgency on actual risk and impact, not just angry wording.

The issue was addressed by adding specific definitions for High, Medium, and Low urgency; general examples for each level; guidance that High reflects safety, health, security, fraud, immediate financial impact, or a serious unresolved issue; guidance that Medium reflects meaningful customer impact without immediate safety, security, or serious financial risk; and guidance that anger alone should not make a complaint High urgency. After updating the prompt and retesting, the model classified complaints more consistently with the project team's expected

labels. The lesson learned is effective prompt engineering, where Generative AI output improves through clear instructions, representative test cases, review of incorrect results, and prompt refinement.

Web Page “Failed to Fetch” Error

A web page “Failed to Fetch” error is a generic JavaScript network error that occurs when a browser tries to request data from a server using the Fetch API but the request is blocked or fails to receive a response (Graham, 2026). During the NovaMart project, the project team submitted fictional customer complaints through the web page, which displayed “Error: Failed to Fetch.” The browser could not complete its request to the API Gateway endpoint, so the complaint did not reach Lambda or Amazon Bedrock for analysis. Troubleshooting revealed that the web page initially used an incomplete API URL and that the API had not been fully deployed to its production stage. Additionally, the API route accepted ANY, allowing the browser’s CORS OPTIONS preflight request to reach Lambda. Because the request contained no complaint, Lambda returned a 400 error.

The project team resolved these issues by deploying the production stage, updating and re-uploading the web page with the correct endpoint, configuring CORS, and restricting the complaint submission route to POST. After redeployment and testing, the web page successfully returned the Claude analysis. This experience reinforced the importance of correctly configuring API endpoints, HTTP methods, and CORS handling when integrating a browser-based interface with a serverless backend.

Multi-Component Code Integration and Time Constraints

The prototype required the project team to integrate code across separate components within a limited timeline: Python code in AWS Lambda for complaint analysis and HTML, CSS, and JavaScript in the S3-hosted index.html file for the user interface. A working AWS application does not use a single block of code; each component has a separate purpose and location. Therefore the project team needed to learn which code belonged in Lambda and the web page, how the web page called the API URL, and how Lambda sent the complaint to Bedrock.

The project team addressed this issue by adapting the reference code for Lambda and the web page, placing each code type in the correct AWS component, testing Lambda with JSON events first, then connecting the web page through the API Gateway URL, and refining the displayed results. This resulted in correct placement and configuration of Lambda and web-page code. The final web page successfully submits a fictional complaint, receives the Lambda/Bedrock result, and displays urgency, sentiment, and draft response. The lesson learned is that AWS applications use separate code and configuration in different components. Testing Lambda with a JSON event before connecting the web page made it easier to identify whether an issue was in the code, API Gateway, CORS, or the user interface.

References

- Chaumond, J. (2020, February 14). *How to Train a New Language Model from Scratch using Transformers and Tokenizers*. Hugging Face. <https://huggingface.co/blog/how-to-train>
- Claude 4.5 Haiku by Anthropic now in Amazon Bedrock*. AWS. (2025, October 15). <https://aws.amazon.com/about-aws/whats-new/2025/10/claude-4-5-haiku-anthropic-amazon-bedrock/>
- Claude Haiku 4.5 - Amazon Bedrock*. AWS. (n.d.-a). <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-anthropic-claude-haiku-4-5.html>
- Claude Haiku 4.5*. Claude Platform Docs. (n.d.-a). <https://platform.claude.com/docs/en/models/haiku-4-5/overview>
- Claude Sonnet 5 - Amazon Bedrock*. AWS. (n.d.-b). <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-anthropic-claude-sonnet-5.html>
- Claude Sonnet 5*. Claude Platform Docs. (n.d.-b). <https://platform.claude.com/docs/en/models/sonnet-5/overview>
- Configure CORS for HTTP APIs in API Gateway*. AWS. (n.d.-c). <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-cors.html>
- Defining Boundaries for Regional Services Access*. AWS. (n.d.-d). <https://docs.aws.amazon.com/whitepapers/latest/navigating-gdpr-compliance/defining-boundaries-for-regional-services-access.html>
- Fregly, C., Barth, A., & Eigenbrode, S. (2023a). *Chapter 1. Generative AI Use Cases, Fundamentals, and Project Life Cycle*. O'Reilly. <https://www.oreilly.com/library/view/generative-ai-on/9781098159214/ch01.html>
- Fregly, C., Barth, A., & Eigenbrode, S. (2023b). *Chapter 5. Fine-Tuning and Evaluation*. O'Reilly. <https://www.oreilly.com/library/view/generative-ai-on/9781098159214/ch05.html>
- Fregly, C., Barth, A., & Eigenbrode, S. (n.d.). *Chapter 7. Fine-Tuning with Reinforcement Learning from Human Feedback*. O'Reilly. <https://www.oreilly.com/library/view/generative-ai-on/9781098159214/>

Google. (n.d.). *Introduction - Text Classification*. Google.

<https://developers.google.com/machine-learning/guides/text-classification>

Google. (n.d.). *Production ML Systems: Monitoring Pipelines*. Google Machine Learning

Education. <https://developers.google.com/machine-learning/crash-course/production-ml-systems/monitoring>

Graham, M. (2026, March). *How to Fix “Network error: failed to fetch” in Claude (Anthropic)*.

RapidDev. <https://www.rapidevelopers.com/ai-build-errors/network-error-failed-to-fetch>

MacDonald, S. (2023, March 16). *Study: 62% of Companies Ignore Customer Service Emails*.

SuperOffice. https://www.superoffice.com/blog/customer-service-benchmark-report/?utm_source=Marketo&utm_medium=email&utm_term=arf&utm_content=MASTHEADLEARNMORE&utm_campaign=Broker_EM8&gclid=CL7_t6DH1dQCFQsDaQodF4gPHw

National Institute of Standards and Technology. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST Technical Series Publications.

<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

Overview - Amazon Bedrock. AWS. (n.d.-e).

<https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html>

Use Amazon S3 Website Hosting to Host without a Single Web Server. AWS. (n.d.-f).

<https://docs.aws.amazon.com/whitepapers/latest/build-static-websites-aws/use-amazon-s3-website-hosting-to-host-without-a-single-web-server.html>

What is Amazon API Gateway?. AWS. (n.d.-g).

<https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>

What is Amazon CloudWatch?. AWS. (n.d.-h).

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html>

What is AWS Lambda? AWS. (n.d.-i).

<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>